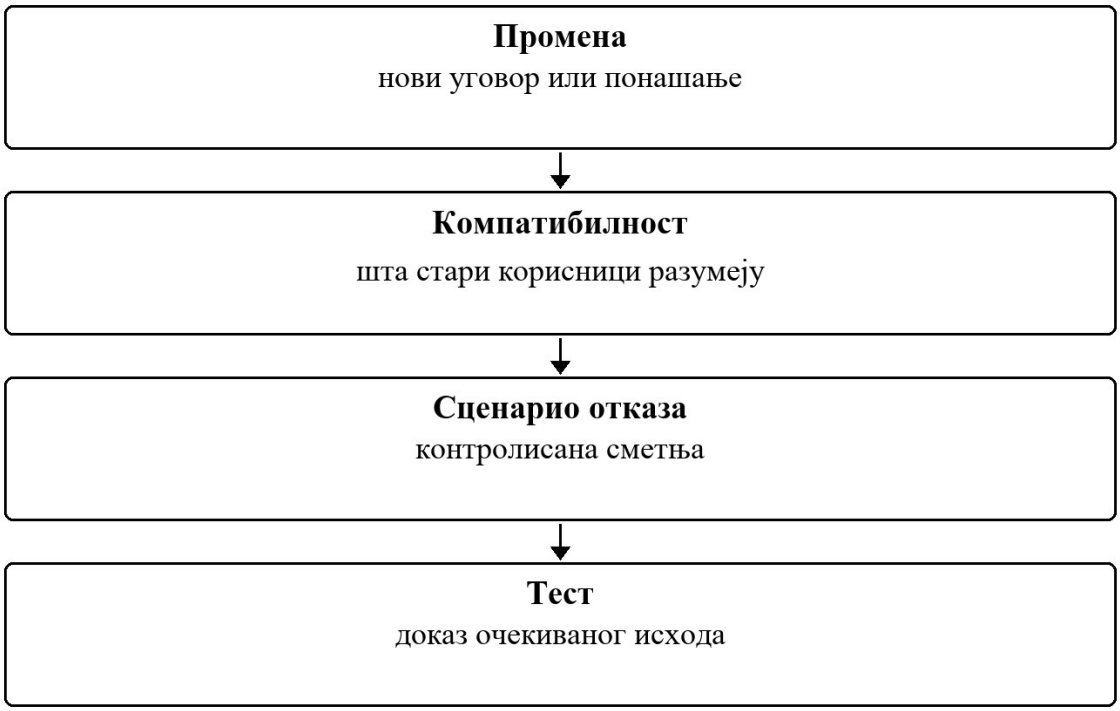
Верзионисање и полазећи од отказа тестирање

Основе дистрибуираног програмирања · 04

У дистрибуираном систему нормалан сценарио никада није цела прича. Порука може да стигне два пута, ван редоследа или након истека рока. Верзије уговора и контролисани отказ сценарији омогућавају тиму да те ситуације проучи док су безбедне и разумљиве.

Полазећи од отказа приступ не предвиђа сваки могући квар. Он подстиче тим да за важну операцију постави права питања: шта ако одговор касни, шта ако се захтев понови и шта ако компонента нестане између две радње?



Слика 1. Развој који рачуна на кашњење, дуплирање и прекиде

Дистрибуирана својства настају у односу између компоненти, времена и граница одговорности.

Фокус вежбе

Материјал објашњава дистрибуирани проблем, последице избора и границе гаранције коју систем може да пружи.

4.1 Неизвесност је део уговора

- Компатибилност се процењује према примаоцима поруке, не само према коду пошиљаоца.
- Кашњење и дупликат су нормални улази које компонента мора да објасни.
- Тест треба да провери видљив исход, не само да метода не баца изузетак.

Питање за дискусију

За коју операцију би касно пристигла порука била опаснија од потпуно изгубљене поруке и зашто?

4.2 Како се ова тема види у развоју

За најважније токове тим формулише неколико малих отказ експеримената и покреће их у истом окружењу. Резултат улази у тестну базу и у опис одлуке, па следећа промена не мора поново да открива иста ограничења.

Честе грешке које треба препознати

- Претпоставка да се порука испоручује тачно једном.
- Некомпатибилна промена означена као безопасна јер локални тестови пролазе.
- Тестирање прекида тек након интеграције свих компоненти.

Веза са остатком градива

Овај принцип се надовезује на уговоре, отказ сценарије, могућност посматрања система и тестирање. Иста логика важи без обзира на конкретну технологију или домен система.

4.3 Пример: потврда стиже након истека рока

Компонента чека одговор до истека рока и покреће безбедан опоравак. Затим стигне стара потврда из првог покушаја. Ако систем нема стабилан идентитет операције и правило за редослед, та потврда може преписати новије стање. Сценарио отказа зато није изузетак који се само ухвати, већ део уговора о понашању.

- Истек времена значи да одговор није стигао на време, не да се радња сигурно није извршила.
- Дупликат и касна порука морају се раздвојити од нове легитимне операције.
- Поновљив симулатор омогућава да се исти редослед грешке провери након сваке измене.

Како се сценарио анализира

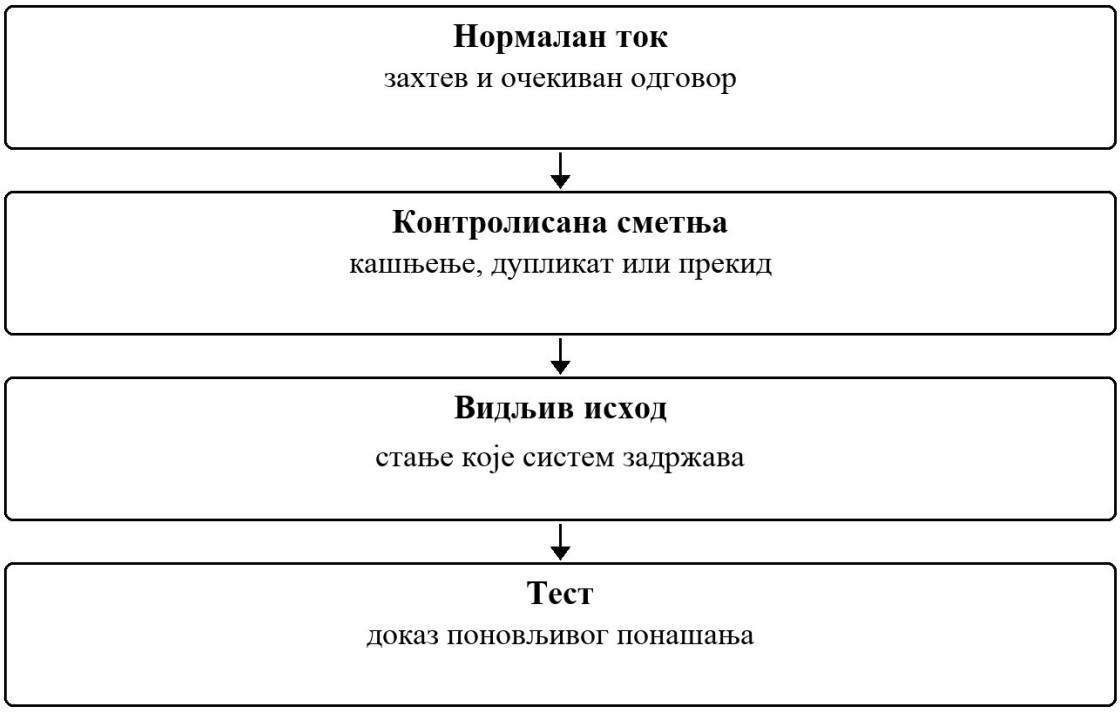
Најпре раздвојите оно што систем зна од онога што претпоставља. Затим одредите стабилан идентитет операције и видљив исход који се може проверити.

4.4 Од принципа до доказа

Дистрибуирани принцип је користан тек када систем има јасан, проверљив одговор на његов сценарио отказа. Добар доказ не показује само да је нормалан ток успео, већ и да кашњење, дупликат, прекид или касна порука не мењају пословно значење операције.

- Именовати стање које је ауторитативно и компоненту која има право да га мења.
- Навести шта се дешава када одговор изостане или стигне након истека рока.
- Понављати контролисани сценарио док се исход може објаснити и независно проверити.

4.5 Шема полазећи од отказа провере



Слика 2. Шема полазећи од отказа провере

Циљ није претпоставити савршен транспорт, већ јасно дефинисати понашање када транспорт није савршен.

Шему користите као кратку проверу: за сваки корак треба умети навести власника одлуке, поруку или стање које се преносе и сценарио отказа који може променити ток.

4.6 Упоредни преглед елемената тока

Табела 1. Елемент и Улога у току

Елемент	Улога у току	Питање које треба поставити
Промена	нови уговор или понашање	Шта се дешава када промена није доступан или даје застарелу информацију?
Компатибилност	шта стари корисници разумеју	Шта се дешава када компатибилност није доступан или даје застарелу информацију?
Сценарио отказа	контролисана сметња	Шта се дешава када сценарио отказа није доступан или даје застарелу информацију?
Тест	доказ очекиваног исхода	Шта се дешава када тест није доступан или даје застарелу информацију?

Поређење појмова

Сваки елемент тока решава други део проблема. Стабилан дизајн не пребацује одговорност једног елемента на други само зато што је то краткорочно једноставније.

4.7 Питања за анализу дистрибуираног понашања

Када се анализира дистрибуирани ток, није довољно описати само срећан пут. Потребно је именовати стање које се мења, границу одговорности, поруку или време које може закаснити и исход који систем мора сачувати упркос неизвесности.

1. Компатибилност се процењује према примаоцима поруке, не само према коду пошиљаоца.

2. Кашњење и дупликат су нормални улази које компонента мора да објасни.

3. Тест треба да провери видљив исход, не само да метода не баца изузетак.

4. Који контролисани сценарио би показао да се пословно значење операције не мења при кашњењу, дупликату или прекиду?

Питање за вежбу

За коју операцију би касно пристигла порука била опаснија од потпуно изгубљене поруке и зашто?

4.8 Извршиви пример

Пример полази од онога што мрежа сигурно ради: иста порука може стићи два пута, може закаснити и може бити послата новијом верзијом уговора. Пријемна страна зато за сваку поруку доноси експлицитну одлуку, а тестови најпре покривају неуспешне путеве.

Пример 1. Покретање демонстрације и тестова

```
cd examples/odp-dotnet/vezba-04-verzionisanje-failure-first
dotnet run --project src/Odp.Vezba04.ConsoleUi
dotnet test Odp.Vezba04.sln
```

Правило у коду

Пример 2. examples/odp-dotnet/vezba-04-verzionisanje-failure-first/src/Odp.Vezba04.Domain/Messaging/InboxPolicy.cs

```
public InboxDecision Decide(MessageEnvelope envelope, bool alreadyProcessed,
DateTimeOffset now)
{
    ArgumentNullException.ThrowIfNull(envelope);

    if (envelope.Version.Major != rules.SupportedMajor)
        return new(false, InboxCodes.UnsupportedMajorVersion);
    if (alreadyProcessed)
        return new(false, InboxCodes.Duplicate);
    if (now - envelope.SentAt > rules.MaxAge)
        return new(false, InboxCodes.TooOld);

    return new(true, InboxCodes.Accepted);
}
```

Тест који задаје квар

Пример 3. examples/odp-dotnet/vezba-04-verzionisanje-failure-first/tests/Odp.Vezba04.Tests/Application/ReceiveMessageHandlerTests.cs

```
[Test]
public void Receive_WhenTheSameMessageIsDeliveredTwice_ProcessesItOnce()
{
    var envelope = Envelope("m-1", 1, Start);

    var first = _handler.Receive(envelope);
    var second = _handler.Receive(envelope);

    using (Assert.EnterMultipleScope())
    {
        Assert.That(first.Code, Is.EqualTo(InboxCodes.Accepted));
        Assert.That(second.Code, Is.EqualTo(InboxCodes.Duplicate));
        Assert.That(_processor.ProcessedPayloads, Has.Count.EqualTo(1));
    }
}
```

Испис демонстрације

Пример 4. дотнет рун --пројецт src/Odp.Vezba04.ConsoleUi

```
m-1 v1.0 (prva isporuka) -> message_accepted
m-1 v1.0 (ponovljena isporuka iste poruke) -> message_duplicate
m-2 v1.3 (novija minor verzija) -> message_accepted
m-3 v2.0 (druga major verzija) -> message_major_version_unsupported
```

```
m-4 v1.0 (poruka koja je putovala 12 minuta) -> message_too_old

Poslovni efekat se dogodio 2 puta:
  telemetrija m-1
  telemetrija m-2
```

Шта треба уочити

- Дупликат, закаснела порука и друга мајор верзија су очекивани улази са сопственим кодом.
- Пословни ефекат се дешава једном, колико год пута да иста порука стигне.
- Новија минор верзија се прихвата; друга мајор верзија се одбија.

Задатак уз пример

Додати правило да се порука са минор верзијом већом од познате прихвата, али се исход означава новим кодом `message_accepted_newer_minor`, како би тим знао да уговор треба прегледати. Покрити гранични случај тестом.

Рад на вежби — примена на пројекат

На додељеној пројектној целини применити принцип из теме „Верзионисање и полазећи од отказа тестирање". Идентификовати конкретну дистрибуирану одлуку, имплементирати је и припремити поновљив сценарио отказа и тест који тим може да покаже на пројектној контролној тачки.

П2 Идентитет, аудит, могућност посматрања система и конфигурација

Контролна тачка

Ова тачка заокружује претходне теме. Организациони детаљи и захтеви налазе се у посебном одељку контролних тачака.